Map Reduce Script

1. Explain the Stages/Entry Points in Map-Reduce Script and Flow of the Stages.

Ans.

Stages in a MapReduce script in NetSuite:

GetInput Stage:

- This is the initial stage where we retrieve the input data
- In NetSuite, we typically define a `getInputData(inputContext)` function, which is responsible for fetching the data we want to process
- This function returns an array of input data that will be passed to the map stage.
- Parameters we have in GetInput Stage:

  1.inputContext.isRestarted - Indicates whether the current invocation of this function is the first Invocation. If true, then this function has been restarted

  2.inputContext.ObjectRef - Object that references the input data

  3.ObjectRef.id - Internal ID of the record instance that contains the input data

  4.ObjectRef.type - Type of the record instance that contains the input data

Map Stage:

- In the map stage, we define a `map(mapContext)` function that processes each piece of input data and emits key-value pairs as intermediate results.
- These key-value pairs are grouped by key during the shuffle stage.
- Parameters we have in Map Stage:

  1.mapContext - Data collection containing the key-value pairs to process in the map stage. This parameter is provided automatically based on the results of the getInputData stage.

  2.mapContext.errors - Serialized errors that were thrown during previous attempts to execute the map

  3.mapContext.executionNo - Number of times the map function has been executed on the current key-value

  4.mapContext.isRestarted - Indicates whether the current invocation of this function is the first invocation if true, then this function has been restarted

  5.mapContext.key - Key to be processed during the map stage

  6.mapContext.value - Value to be processed during the map stage

Shuffle Stage:

- NetSuite's MapReduce framework handles the shuffle phase internally.
- It groups and sorts the intermediate key-value pairs produced by the map stage.
- We don't explicitly define this stage in script, but it's an important part of the MapReduce process.

Reduce Stage:

- After the shuffle phase, NetSuite automatically sorts and groups the data by key and passes it to `reduce` function.
- In the reduce stage, we define how to process and aggregate the data with the same key.
- Parameters we have in Reduce Stage:

  1.reduceContext - Data collection containing the groups to process in the reduce stage. This parameter is provided automatically based on the results of the map stage.

  2.reduceContext.errors - Serialized errors that were thrown during previous attempts to execute the

  3. reduceContext.executionNo - Number of times the reduce function has been executed on the current group

  4. reduceContext.isRestarted - Indicates whether the current invocation of this function is the first invocation if true, the current invocation is not the first invocation, and this function has been restarted

  5. reduceContext.key - Key to be processed during the reduce stage

  6. reduceContext.values - All values associated with a unique key that was passed to the reduce stage for processing

Summarize Stage:

- In NetSuite MapReduce, there is often a "summarize" step after the reduce stage.
- This step allows us to perform final calculations or output the results in the desired format.
- We define a `summarize` function for this purpose, which processes the output of the reduce stage and performs any necessary final operations.

  1.summaryContext - Statistics about the execution of a map/reduce script

  2.summaryContext.concurrency - Maximum concurrency number when executing parallel tasks for the map/reduce

  3.summaryContext.dateCreated - The date and time when the map/reduce script began running

4.summaryContext.isRestarted - Indicates whether the current invocation of this function is the first invocation (if true, the current invocation is not the first invocation, and this function has been restarted)

5.summaryContext.output - Serialized keys and values that were saved as output during the reduce stage

6.summaryContext.seconds - Total seconds elapsed when running the map/reduce script

7.summaryContext.usage - Total number of governance usage units consumed when running the map/reduce

8.summaryContext.yields - Total number of yields when running the map/reduce script

9.summaryContext.inputSummary - Statistics about the input stage

10.summaryContext.mapSummary - Statistics about the map stage

11.summaryContext.reduceSummary - Statistics about the reduce stage

Flow of the Stages:

GetInput Stage is the first stage where you define a `getInputData` function. NetSuite retrieves the input data using this function, and it's passed as an array of input records to the subsequent stages. After the input data is obtained, the MapReduce framework processes it through the `map` function that we define. The `map` function operates on each input record, extracting key-value pairs as intermediate results. NetSuite's MapReduce framework automatically handles the shuffle and sort phase internally. It groups and sorts the intermediate key-value pairs produced by the map stage based on their keys. This is a critical step for grouping and preparing data for the reduce stage. After shuffling and sorting, the grouped data is passed to the `reduce` function that you've defined. In the `reduce` function, we specify how to process and aggregate the data with the same key. This stage can involve complex data transformation and summarization. Once the reduce stage is complete, NetSuite provides a `summarize` function that you can define. In the `summarize` function, we can perform any final calculations, generate reports, or format the results as needed. This stage is often used for generating the final output of the MapReduce job.

2. Write Down any Two Scenario where Map-Reduce script is Used.

Ans.

Social Media Sentiment Analysis:

- Social media platforms generate vast amounts of text data. MapReduce can be employed to analyze this data for sentiment analysis, helping companies understand public sentiment towards their products or brands.
- In the "map" stage, text data is processed, and in the "reduce" stage, sentiment scores are aggregated to provide an overall sentiment analysis report.

Large-scale Image Processing:

- When dealing with a massive collection of images, MapReduce can be used for tasks like image recognition, object detection, or generating image thumbnails.
- Images are divided into chunks, and the "map" function processes them in parallel. In the "reduce" stage, results can be combined or further analyzed.

Fraud Detection in Financial Transactions:

- Financial institutions can utilize MapReduce to detect fraudulent activities by analyzing transaction data.
- The "map" stage can identify suspicious patterns or anomalies in individual transactions, and the "reduce" stage can consolidate the results to flag potential fraud cases.

3. Discuss Yeilding, buffer size, exitOnError , retryCount in Map-Reduce Script.

Ans.

Yielding:

- In NetSuite MapReduce scripts, we can use the `yield` keyword or function to pause the execution of the current Map or Reduce function and release control back to the NetSuite server.
- This can be helpful when dealing with potentially long-running tasks or to allow other scripts to execute in a shared environment. Yielding helps prevent script timeouts and ensures that other processes can run.
- This value affects the map and reduces jobs that are associated with an instance of this script deployment. If a single map or reduce job occupies a processor for longer than the limit our entry in this field, the system may force the job to yield. In this case, the system automatically reschedules the work for later. We can enter a value of 3 to 60 in this field.

Buffer Size:

- The `BufferSize` parameter is used to specify the size of the buffer in terms of the number of records to be processed at once. This parameter can be set when configuring our MapReduce script.
- It determines how many records are processed in a single batch before being handed off to the map or reduce function. Setting an appropriate buffer size can help optimize script performance.
- A value that indicates how many key/value pairs a map or reduce job can process before information about the job's progress is saved to the database. Setting this field to a higher number can be more efficient in certain cases.
- However, a low buffer size value minimizes the risk of any pairs being processed twice, which can lead to data duplication if the script is interrupted. In general, leave this value set to 1 unless we have special circumstances that dictate otherwise.

ExitOnError:

- The `ExitOnError` parameter allows us to control whether the MapReduce script continues processing when it encounters an error.
- If set to `true`, the script will exit immediately when an error is encountered. If set to `false`, the script will continue processing and log the error, which can be useful for identifying issues while still processing the data.

RetryCount:

- In a NetSuite MapReduce script, we can use the `retryCount` parameter to specify the number of times the system should attempt to execute the script in the event of a failure or an unhandled error.
- If the specified number of retries is reached, the script can be marked as failed. This parameter is helpful for managing script reliability in case of transient issues.

4. Create a Map-Reduce Script which is used to group the invoice by customer and send the email to the customer.

Ans.

- Here in map reduce script, we will use saved search to group invoice with respect to customer
- In getInputData we will use search.create() to create search of invoice
- And we have set filters for type as a customer invoice and transaction date as from 01 November to 16 November 2023
- search result column will return to map () function
- We will parse mapContext.value to fetch customer name, id, invoice id etc.
- These values will return to reduce using mapContext.write()
- Again, values will parse from reduceContext.values
- Here we will use for loop to add invoice id, date and amount to the mail body
- And hence, we will send invoice details in mail to customer using email.send()

1.Map-Reduce script:

```
/**
* @NApiVersion 2.1
* @NScriptType MapReduceScript
*/
define(['N/record','N/runtime','N/search','N/email'],
    /**
* @param{record} record
*/
    (record,runtime,search,email) => {
        const getInputData = (inputContext) => {
            var searchInvoice_obj = search.create({
            type: "transaction",
            filters:
            [
                ["type", "anyof", "CustInvc"],
                        "AND",
                ["mainline", "is", true],
                        "AND",
                ["trandate", "within", "01 November, 2023" ,"16 November, 2023"]
            ],
            columns:
            [
            search.createColumn({name: "internalid", label: "Internal Id"}),
            search.createColumn({name: "ordertype",sort: search.Sort.ASC,label: "Order Type"}),
            search.createColumn({name: "mainline", label: "*"}),
            search.createColumn({name: "trandate", label: "Date"}),
            search.createColumn({name: "type", label: "Type"}),
            search.createColumn({name: "entity", label: "Name"}),
            search.createColumn({name: "amount", label: "Amount"}),
            search.createColumn({name: "tranid", label: "Document Number" }),

            ]
            });
```

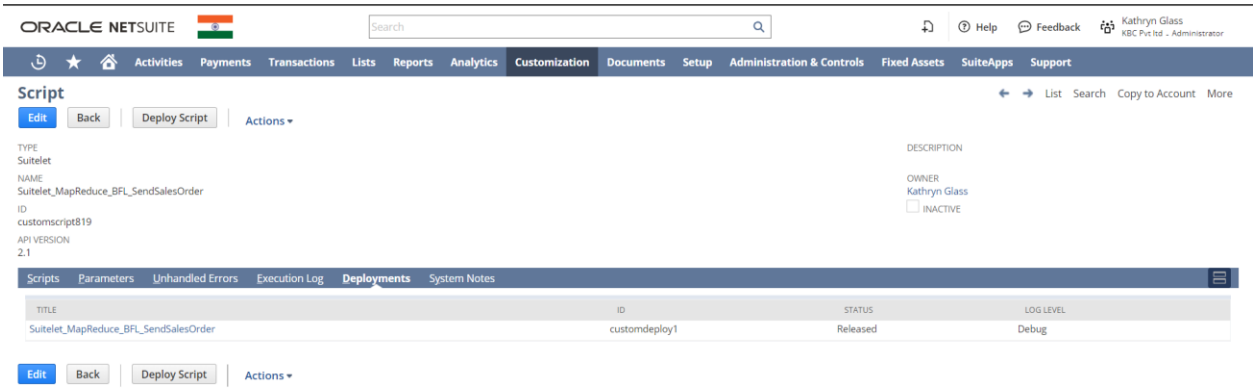
```
        ]
        });
        var searchInvoice_result = searchInvoice_obj.run().getRange(0,25);

        return searchInvoice_result;
    }

    const map = (mapContext) => {

        var map_key = JSON.parse(mapContext.key);

        var map_value = JSON.parse(mapContext.value);

        var customer_id = map_value.values.entity[0].value;

        var customer_name = map_value.values.entity[0].text;

        var invoice_id = map_value.values.tranid;

        var invoice_date = map_value.values.trandate;

        var invoice_amount = map_value.values.amount;

        mapContext.write({
            key : customer_id ,
            value : {
                'custpage_customer_id' : customer_id,
                'custpage_customer_name' : customer_name,
                'custpage_invoice_id' : invoice_id,
                'custpage_invoice_date' : invoice_date,
                'custpage_invoice_amount' : invoice_amount,
            }
```

```
        mapContext.write({
            key : customer_id ,
            value : {
                'custpage_customer_id' : customer_id,
                'custpage_customer_name' : customer_name,
                'custpage_invoice_id' : invoice_id,
                'custpage_invoice_date' : invoice_date,
                'custpage_invoice_amount' : invoice_amount,
            }
        })

    }
    const reduce = (reduceContext) => {

        var reduce_key = reduceContext.key;

        var reduce_value = reduceContext.values;

        var customer_id = JSON.parse(reduceContext.values[0]).custpage_customer_id;

        log.debug('customer_id : ' , customer_id );

        var customer_name = JSON.parse(reduceContext.values[0]).custpage_customer_name;

        log.debug('customer_name : ' , customer_name );

        var email_subject = "Hello , " + customer_name + "  Greetings from Blueflame Labs Pvt.Ltd";

        var email_body = "Here are your invoices of November 2023   :    ";

        email_body+= ' Name : ' + customer_name + ' , ID : ' +  customer_id + '  |  ';

        for(var i = 0 ; i< reduce_value.length ; i++){
```

```
        email_body+= ' Name : ' + customer_name + ' , ID : ' +  customer_id + '  |  ';

        for(var i = 0 ; i< reduce_value.length ; i++){

            var customer_invoice = JSON.parse(reduceContext.values[i]);

            var invoice_id = customer_invoice.custpage_invoice_id;

            var invoice_date = customer_invoice.custpage_invoice_date;

            var invoice_amount = customer_invoice.custpage_invoice_amount;

            email_body += '     Invoice ID : ' + invoice_id + ' , Date : '+ invoice_date + ' , Amount : ' + invoice_amount + '   |  '


        }

        log.debug('email body : ' , email_body);

        email.send({
            author: -5,
            recipients: customer_id,
            subject: email_subject,
            body: email_body
        });

        log.debug('mail sent', 'mail sent')


    }
    const summarize = (summaryContext) => {
    }
```

```
            var customer_invoice = JSON.parse(reduceContext.values[i]);

            var invoice_id = customer_invoice.custpage_invoice_id;

            var invoice_date = customer_invoice.custpage_invoice_date;

            var invoice_amount = customer_invoice.custpage_invoice_amount;

            email_body += '     Invoice ID : ' + invoice_id + ' , Date : '+ invoice_date + ' , Amount : ' + invoice_amount + '    |  '


        }

        log.debug('email body : ' , email_body);

        email.send({
            author: -5,
            recipients: customer_id,
            subject: email_subject,
            body: email_body
        });

        log.debug('mail sent', 'mail sent')


    }
    const summarize = (summaryContext) => {
    }
    return {getInputData, map, reduce, summarize}
});
```

2.Script has been deployed:



3.Execution log of script:

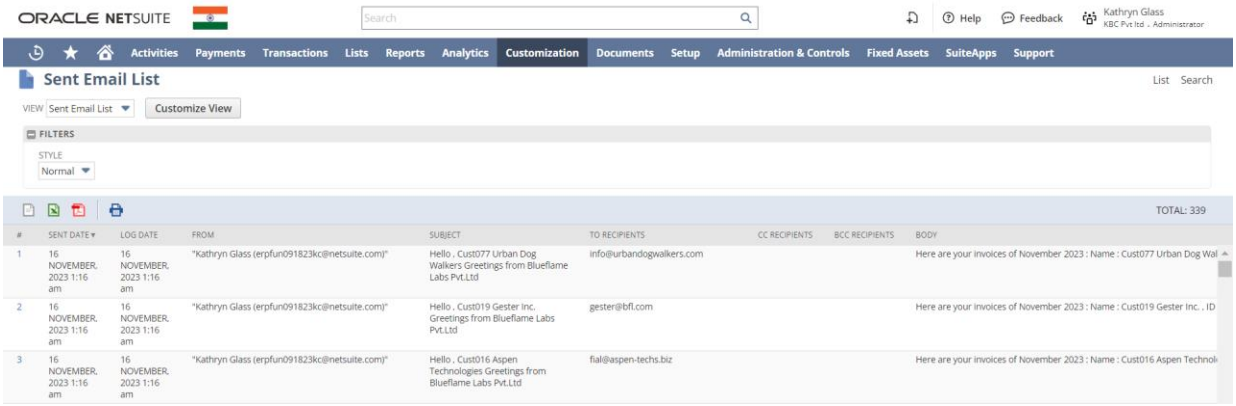
VIEW: Default Script Notes View | TYPE: - All -

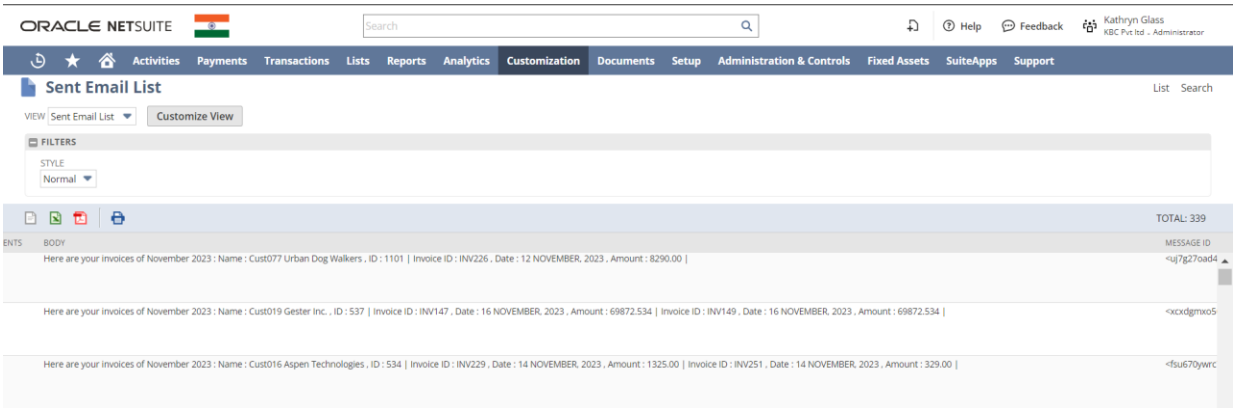
Customize View | Remove all | Refresh

#	VIEW	TYPE	TITLE	DATE	TIME	USER	DETAILS	REMOVE
1	View	Debug	mail sent	16 NOVEMBER, 2023	1:16 am	- System-	mail sent	Remove
2	View	Debug	email body :	16 NOVEMBER, 2023	1:16 am	- System-	Here are your invoices of November 2023 : Name : Cust019 Gester Inc. , ID : 537 | Invoice ID : INV147 , Date : 16 NOVEMBER, 2023 , Amount : 69872.534 | Invoice ID : INV149 , Date : 16 NOVEMBER, 2023 , Amount : 69872.534 |	Remove
3	View	Debug	customer_name :	16 NOVEMBER, 2023	1:16 am	- System-	Cust019 Gester Inc.	Remove
4	View	Debug	customer_id :	16 NOVEMBER, 2023	1:16 am	- System-	537	Remove
5	View	Debug	mail sent	16 NOVEMBER, 2023	1:16 am	- System-	mail sent	Remove
6	View	Debug	email body :	16 NOVEMBER, 2023	1:16 am	- System-	Here are your invoices of November 2023 : Name : Cust016 Aspen Technologies , ID : 534 | Invoice ID : INV229 , Date : 14 NOVEMBER, 2023 , Amount : 1325.00 | Invoice ID : INV251 , Date : 14 NOVEMBER, 2023 , Amount : 329.00 |	Remove
7	View	Debug	customer_name :	16 NOVEMBER, 2023	1:16 am	- System-	Cust016 Aspen Technologies	Remove
8	View	Debug	customer_id :	16 NOVEMBER, 2023	1:16 am	- System-	534	Remove
9	View	Debug	mail sent	16 NOVEMBER, 2023	1:16 am	- System-	mail sent	Remove
10	View	Debug	email body :	16 NOVEMBER, 2023	1:16 am	- System-	Here are your invoices of November 2023 : Name : Cust077 Urban Dog Walkers , ID : 1101 | Invoice ID : INV226 , Date : 12 NOVEMBER, 2023 , Amount : 8290.00 |	Remove
11	View	Debug	customer_name :	16 NOVEMBER, 2023	1:16 am	- System-	Cust077 Urban Dog Walkers	Remove
12	View	Debug	customer_id :	16 NOVEMBER, 2023	1:16 am	- System-	1101	Remove

Edit | Back | Deploy Script | Actions

4.Email has sent to respective customer:





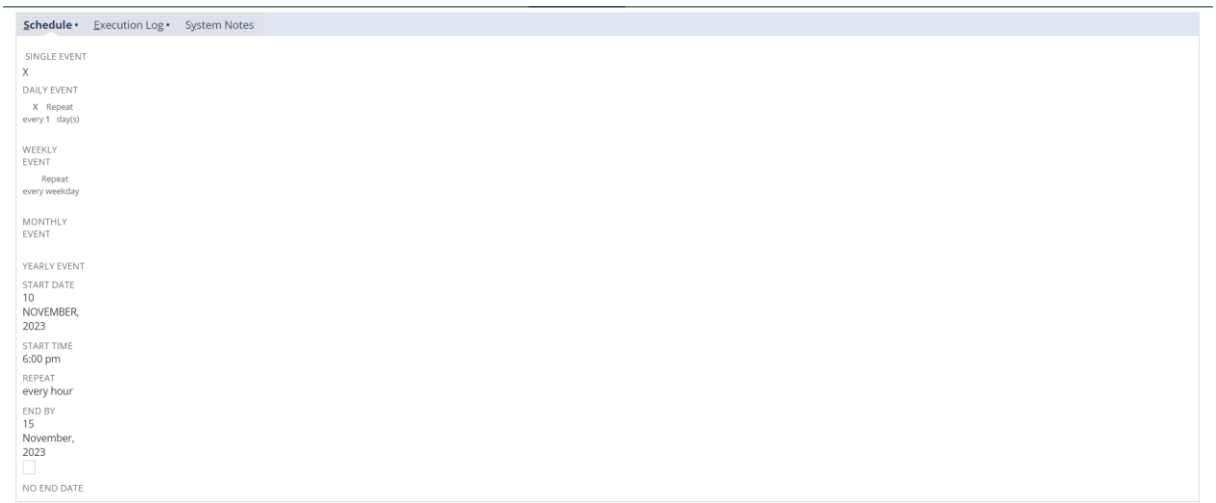
5. Schedule a Map-Reduce to run once after 1 hour on for each day and attach Screenshot.

Ans.

- Here we will use new Date () to print the time of script scheduling.
- And log.debug() to shedule script for every hour.

```
/**
 * @NApiVersion 2.1
 * @NScriptType MapReduceScript
 */
define(['N/currentRecord'],
    (currentRecord) => {

        const getInputData = (inputContext) => {
            var now = new Date();

            log.debug('Map Reduce Script has been shedule for each hour Trigger Time : ' + now );
            log.debug('Current Trigger Time : ' + now );

        }

        const map = (mapContext) => {

        }

        const reduce = (reduceContext) => {

        }

        const summarize = (summaryContext) => {

        }

        return {getInputData, map, reduce, summarize}

});
```

- Script has been shedule



- Log has printed after every hour; hence script has been shedule for each hour.

TYPE
Map/Reduce

NAME
BFL | MapReduce_SheduleforEachHour

ID
customscript818

API VERSION
2.1

DESCRIPTION

OWNER
Kathryn Glass

INACTIVE

Scripts | Parameters | Unhandled Errors | **Execution Log** | Deployments | System Notes

VIEW: Default Script Notes View
TYPE: - All -

Customize View | Remove all | Refresh

#	VIEW	TYPE	TITLE	DATE	TIME	USER	DETAILS	REMOVE
1	View	Debug	Current Trigger Time : Mon Nov 13 2023 23:03:25 GMT-0800 (PST)	13 NOVEMBER, 2023	11:03 pm	-System-		Remove
2	View	Debug	Map Reduce Script has been shedule for each hour Trigger Time : Mon Nov 13 2023 23:03:25 GMT-0800 (	13 NOVEMBER, 2023	11:03 pm	-System-		Remove
3	View	Debug	Current Trigger Time : Mon Nov 13 2023 22:03:08 GMT-0800 (PST)	13 NOVEMBER, 2023	10:03 pm	-System-		Remove
4	View	Debug	Map Reduce Script has been shedule for each hour Trigger Time : Mon Nov 13 2023 22:03:08 GMT-0800 (	13 NOVEMBER, 2023	10:03 pm	-System-		Remove
5	View	Debug	Current Trigger Time : Mon Nov 13 2023 21:03:21 GMT-0800 (PST)	13 NOVEMBER, 2023	9:03 pm	-System-		Remove
6	View	Debug	Map Reduce Script has been shedule for each hour Trigger Time : Mon Nov 13 2023 21:03:21 GMT-0800 (	13 NOVEMBER, 2023	9:03 pm	-System-		Remove
7	View	Debug	Current Trigger Time : Mon Nov 13 2023 20:03:18 GMT-0800 (PST)	13 NOVEMBER, 2023	8:03 pm	-System-		Remove
8	View	Debug	Map Reduce Script has been shedule for each hour Trigger Time : Mon Nov 13 2023 20:03:18 GMT-0800 (	13 NOVEMBER, 2023	8:03 pm	-System-		Remove
9	View	Debug	Current Trigger Time : Mon Nov 13 2023 19:07:04 GMT-0800 (PST)	13 NOVEMBER, 2023	7:07 pm	-System-		Remove
10	View	Debug	Map Reduce Script has been shedule for each hour Trigger Time : Mon Nov 13 2023 19:07:04 GMT-0800 (	13 NOVEMBER, 2023	7:07 pm	-System-		Remove
11	View	Debug	Current Trigger Time : Mon Nov 13 2023 18:04:38 GMT-0800 (PST)	13 NOVEMBER, 2023	6:04 pm	-System-		Remove
12	View	Debug	Map Reduce Script has been shedule for each hour Trigger Time : Mon Nov 13 2023 18:04:38 GMT-0800 (	13 NOVEMBER, 2023	6:04 pm	-System-		Remove

6. Create a Map-Reduce script that sends a single email to every customer with at least one past due invoice. The email will contain a list of those invoices with some basic information about them.

Ans.

- Here in map reduce script, we will use saved search to group invoice with respect to customer
- In getInputData we will use search.create() to create search of invoice
- we have set filters for type as a customer invoice and transaction date as from 01 November to 16 November 2023
- And, we have filter of billingstatus as a true to fetch the invoices of pending payment
- search result column will return to map () function
- We will parse mapContext.value to fetch customer name, id, invoice id etc.
- These values will return to reduce using mapContext.write()
- Again, values will parse from reduceContext.values
- Here we will use for loop to add invoice id, date and amount to the mail body
- And hence, we will send invoice details in mail to customer using email.send()

1.Map-Reduce script:

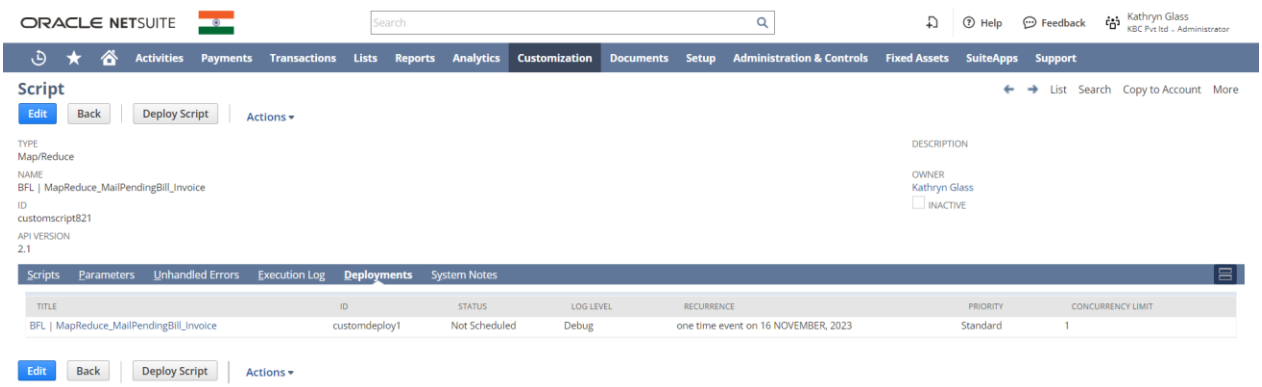
```
/**
* @NApiVersion 2.1
* @NScriptType MapReduceScript
*/
define(['N/record', 'N/runtime', 'N/search', 'N/email'],
    /**
* @param{record} record
*/
    (record, runtime, search, email) => {
        const getInputData = (inputContext) => {
            var searchInvoice_obj = search.create({
                type: "transaction",
                filters:
                    [

                        ["mainline", "is", "T"],
                        "AND",
                        ["type", "anyof", "CustInvc"],
                        "AND",
                        ["trandate", "within", "01 November, 2023", "16 November, 2023"],
                        "AND",
                        ["billingstatus", "is", "T"]
                    ],
                columns:
                    [
                        search.createColumn({ name: "internalid", label: "Internal Id" }),
                        search.createColumn({ name: "ordertype", sort: search.Sort.ASC, label: "Order Type" }),
                        search.createColumn({ name: "mainline", label: "*" }),
                        search.createColumn({ name: "trandate", label: "Date" }),
                        search.createColumn({ name: "type", label: "Type" }),
                        search.createColumn({ name: "entity", label: "Name" }),
                        search.createColumn({ name: "amount", label: "Amount" }),
                        search.createColumn({ name: "tranid", label: "Document Number" }),
                        search.createColumn({name: "statusref", label: "Status"}),


                    ]
            });
            var searchInvoice_result = searchInvoice_obj.run().getRange(0, 25);
```

```
                search.createColumn({name: "statusref", label: "Status"}),


            ]
        });
        var searchInvoice_result = searchInvoice_obj.run().getRange(0, 25);

        return searchInvoice_result;
    }

    const map = (mapContext) => {

        var map_key = JSON.parse(mapContext.key);

        var map_value = JSON.parse(mapContext.value);

        var customer_id = map_value.values.entity[0].value;

        var customer_name = map_value.values.entity[0].text;

        var invoice_id = map_value.values.tranid;

        var invoice_date = map_value.values.trandate;

        var invoice_amount = map_value.values.amount;

        var invoice_status = map_value.values.statusref[0].value;

        mapContext.write({
            key: customer_id,
            value: {
                'custpage_customer_id': customer_id,
                'custpage_customer_name': customer_name,
                'custpage_invoice_id': invoice_id,
                'custpage_invoice_date': invoice_date,
                'custpage_invoice_amount': invoice_amount,
                'custpage_invoice_status': invoice_status,
            }
        })
```

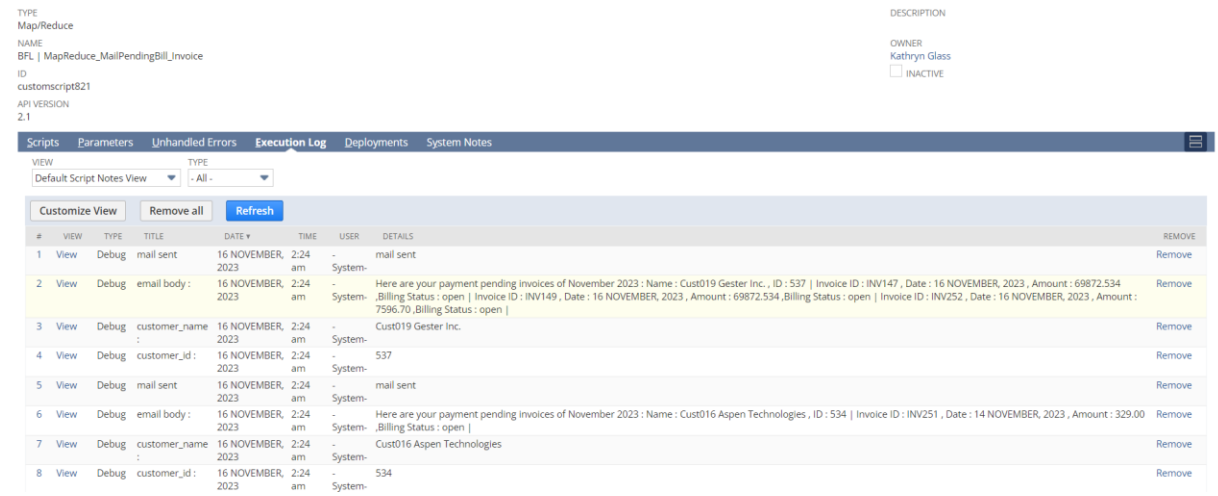
```
        mapContext.write({
            key: customer_id,
            value: {
                'custpage_customer_id': customer_id,
                'custpage_customer_name': customer_name,
                'custpage_invoice_id': invoice_id,
                'custpage_invoice_date': invoice_date,
                'custpage_invoice_amount': invoice_amount,
                'custpage_invoice_status': invoice_status,
            }
        })
        log.debug('map' , map_value)

    }
    const reduce = (reduceContext) => {

        var reduce_key = reduceContext.key;

        var reduce_value = reduceContext.values;

        var customer_id = JSON.parse(reduceContext.values[0]).custpage_customer_id;

        log.debug('customer_id : ', customer_id);

        var customer_name = JSON.parse(reduceContext.values[0]).custpage_customer_name;

        log.debug('customer_name : ', customer_name);

        var email_subject = "Hello , " + customer_name + "  Greetings from Blueflame Labs Pvt.Ltd";

        var email_body = "Here are your payment pending invoices of November 2023   :    ";

        email_body += ' Name : ' + customer_name + ' , ID : ' + customer_id + '  |  ';

        for (var i = 0; i < reduce_value.length; i++) {

            var customer_invoice = JSON.parse(reduceContext.values[i]);
```

```
        email_body += ' Name : ' + customer_name + ' , ID : ' + customer_id + '  |  ';
        for (var i = 0; i < reduce_value.length; i++) {
            var customer_invoice = JSON.parse(reduceContext.values[i]);
            var invoice_id = customer_invoice.custpage_invoice_id;
            var invoice_date = customer_invoice.custpage_invoice_date;
            var invoice_amount = customer_invoice.custpage_invoice_amount;
            var invoice_status = customer_invoice.custpage_invoice_status;
            email_body += '     Invoice ID : ' + invoice_id + ' , Date : ' + invoice_date + ' , Amount : ' + invoice_amount + ' ,Billing Status : ' +invoice_sta

        }
        log.debug('email body : ', email_body);
        email.send({
            author: -5,
            recipients: customer_id,
            subject: email_subject,
            body: email_body
        });
        log.debug('mail sent', 'mail sent')


    }
    const summarize = (summaryContext) => {
    }
    return { getInputData, map, reduce, summarize }
});
```

2.Script has been deployed:



3.Log of script:



TYPE
Map/Reduce
NAME
BFL | MapReduce_MailPendingBill_Invoice
ID
customscript821
API VERSION
2.1

DESCRIPTION
OWNER
Kathryn Glass
INACTIVE

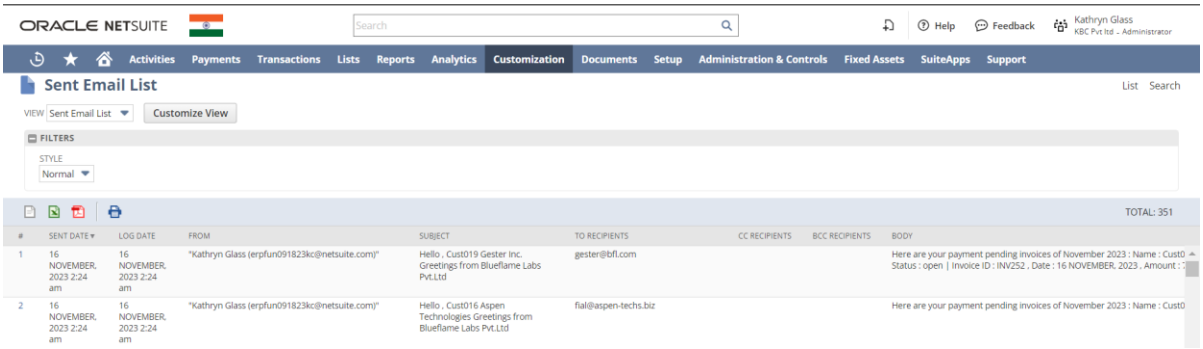
Scripts | Parameters | Unhandled Errors | **Execution Log** | Deployments | System Notes

VIEW: Default Script Notes View
TYPE: - All -

Customize View | Remove all | Refresh

#	VIEW	TYPE	TITLE	DATE	TIME	USER	DETAILS	REMOVE
1	View	Debug	mail sent	16 NOVEMBER, 2023	2:24 am	- System-	mail sent	Remove
2	View	Debug	email body :	16 NOVEMBER, 2023	2:24 am	- System-	Here are your payment pending invoices of November 2023 : Name : Cust019 Gester Inc. , ID : 537 | Invoice ID : INV147 , Date : 16 NOVEMBER, 2023 , Amount : 69872.534 ,Billing Status : open | Invoice ID : INV149 , Date : 16 NOVEMBER, 2023 , Amount : 69872.534 ,Billing Status : open | Invoice ID : INV252 , Date : 16 NOVEMBER, 2023 , Amount : 7596.70 ,Billing Status : open |	Remove
3	View	Debug	customer_name :	16 NOVEMBER, 2023	2:24 am	- System-	Cust019 Gester Inc.	Remove
4	View	Debug	customer_id :	16 NOVEMBER, 2023	2:24 am	- System-	537	Remove
5	View	Debug	mail sent	16 NOVEMBER, 2023	2:24 am	- System-	mail sent	Remove
6	View	Debug	email body :	16 NOVEMBER, 2023	2:24 am	- System-	Here are your payment pending invoices of November 2023 : Name : Cust016 Aspen Technologies , ID : 534 | Invoice ID : INV251 , Date : 14 NOVEMBER, 2023 , Amount : 329.00 ,Billing Status : open |	Remove
7	View	Debug	customer_name :	16 NOVEMBER, 2023	2:24 am	- System-	Cust016 Aspen Technologies	Remove
8	View	Debug	customer_id :	16 NOVEMBER, 2023	2:24 am	- System-	534	Remove

4.Email has been sent to customer of pending payment invoices :



ORACLE NETSUITE | Search | Help | Feedback | Kathryn Glass, KBC Pvt ltd - Administrator

Activities | Payments | Transactions | Lists | Reports | Analytics | Customization | Documents | Setup | Administration & Controls | Fixed Assets | SuiteApps | Support

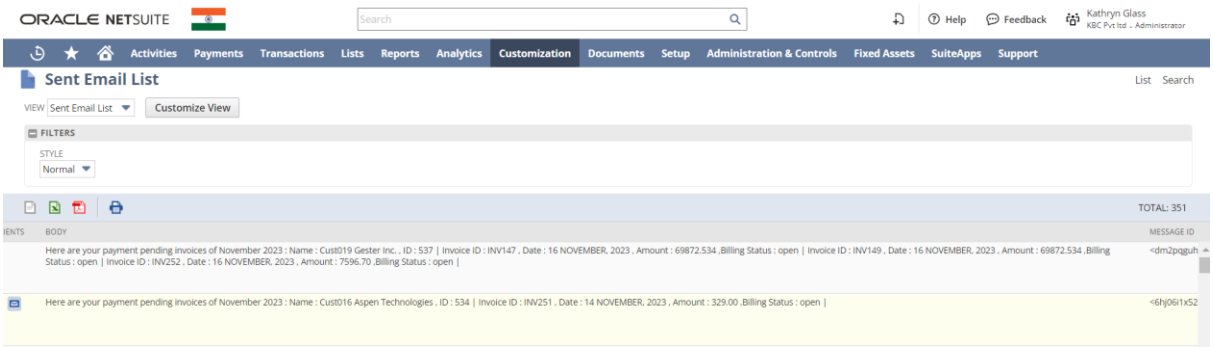
Sent Email List — List | Search

VIEW: Sent Email List | Customize View

FILTERS
STYLE: Normal

TOTAL: 351

#	SENT DATE	LOG DATE	FROM	SUBJECT	TO RECIPIENTS	CC RECIPIENTS	BCC RECIPIENTS	BODY
1	16 NOVEMBER, 2023 2:24 am	16 NOVEMBER, 2023 2:24 am	"Kathryn Glass (erpfun091823kc@netsuite.com)"	Hello , Cust019 Gester Inc. Greetings from Blueflame Labs Pvt.Ltd	gester@bfl.com			Here are your payment pending invoices of November 2023 : Name : Cust0 Status : open | Invoice ID : INV252 , Date : 16 NOVEMBER, 2023 , Amount : :
2	16 NOVEMBER, 2023 2:24 am	16 NOVEMBER, 2023 2:24 am	"Kathryn Glass (erpfun091823kc@netsuite.com)"	Hello , Cust016 Aspen Technologies Greetings from Blueflame Labs Pvt.Ltd	fial@aspen-techs.biz			Here are your payment pending invoices of November 2023 : Name : Cust0



ORACLE NETSUITE | Search | Help | Feedback | Kathryn Glass, KBC Pvt ltd - Administrator

Activities | Payments | Transactions | Lists | Reports | Analytics | Customization | Documents | Setup | Administration & Controls | Fixed Assets | SuiteApps | Support

Sent Email List — List | Search

VIEW: Sent Email List | Customize View

FILTERS
STYLE: Normal

TOTAL: 351

IENTS	BODY	MESSAGE ID
	Here are your payment pending invoices of November 2023 : Name : Cust019 Gester Inc. , ID : 537 | Invoice ID : INV147 , Date : 16 NOVEMBER, 2023 , Amount : 69872.534 ,Billing Status : open | Invoice ID : INV149 , Date : 16 NOVEMBER, 2023 , Amount : 69872.534 ,Billing Status : open | Invoice ID : INV252 , Date : 16 NOVEMBER, 2023 , Amount : 7596.70 ,Billing Status : open |	<dm2pqguh
	Here are your payment pending invoices of November 2023 : Name : Cust016 Aspen Technologies , ID : 534 | Invoice ID : INV251 , Date : 14 NOVEMBER, 2023 , Amount : 329.00 ,Billing Status : open |	<6hj06i1x52

Additional Question

Create a suitelet with body field as filter:

- Customer.
- From date.
- To date.
- Location.

Based on these values filter the sales order and display on the suitelet sublist. Sublist columns would be:

- Checkbox
- Invoice Number
- Customer Name
- Date
- Amount

Selected Invoices from the sublist, get the ids of invoice and send it to:

- Schedule
- Map/Reduce

And update the memo field with message as “Update from Schedule or Map/Reduce.”
Ans:

Suitelet Script:

1.Create a suitelet form using serverWidget.createForm()
2.Add field in the form and link suitelet with client script
3.Enter values in field
4.Get request parameter in POST
5.use response.sendRedirect() to display search order list in same page of suitelet
6.Send parameters to GET
7.Fetch parameters from POST
8.If parameters are true then search sales order according to enter values
9.Checked checkbox of sales order which we want to set memo
10.click on set memo custom button

Client Script:

1.Creat an array to save sales order id
2.In fieldChanged, if changed field is checkbox
3.then take sales order id of that line
4.push id in array
5.create a setMemo function
6.append array of ids in url of suitelet
7.and do windows.open

Suitelet Script:

1.Get array from url using request.parameter
2.Create a task to call scheduledScript
3.Set array in Schedule script parameter

Schedule Script:

1.Get array from parameter
2.load sales order for each internal id using record.load()
3.and set memo for check sales order

1.Suitelet Script

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
define(['N/currentRecord', 'N/ui/serverWidget', 'N/url', 'N/https', 'N/task', 'N/search'],

    (currentRecord, serverWidget, url, http, task, search) => {

        function onRequest(scriptContext) {
            if (scriptContext.request.method === 'GET') {

                const customerName = scriptContext.request.parameters.custpage_customer_name;
                const fromDate = scriptContext.request.parameters.custpage_from_date;
                const toDate = scriptContext.request.parameters.custpage_to_date;
                const SO_Array = scriptContext.request.parameters.custpage_arr;


                let form = serverWidget.createForm({
                    title: 'Sales Order'
                });
                form.clientScriptFileId =  8800;



                var primary = form.addFieldGroup({
                    id: 'primary',
                    label: 'Primary Information'
                });
```

```
});

var secondary = form.addFieldGroup({
    id: 'secondary',
    label: 'Duration '
});

var customerField = form.addField({
    id: 'custpage_customer_name',
    type: serverWidget.FieldType.SELECT,
    label: 'Customer',
    source: 'customer',
    container: 'primary'
});

if(customerName){
    customerField.defaultValue = customerName
}

form.addField({
    id: 'custpage_location',
    type: serverWidget.FieldType.SELECT,
    label: 'Location',
    source: 'location',
    container: 'primary'

});
```

```
var fromField = form.addField({
    id: 'custpage_from_date',
    type: serverWidget.FieldType.DATE,
    label: 'From',
    container: 'secondary'


});

if(fromDate){
    fromField.defaultValue = fromDate
}

var toField = form.addField({
    id: 'custpage_to_date',
    type: serverWidget.FieldType.DATE,
    label: 'To',
    container: 'secondary'


});

if(toDate){
    toField.defaultValue = toDate
}

var sales_sublist = form.addSublist({
    id: 'sales_order_sublist',
```

```
        label: 'To',
        container: 'secondary'


    });

    if(toDate){
        toField.defaultValue = toDate
    }

    var sales_sublist = form.addSublist({
        id: 'sales_order_sublist',
        type: serverWidget.SublistType.LIST,
        label: 'Sales Orders'
    })
    sales_sublist.addField({ id: "custpage_checkbox", type: serverWidget.FieldType.CHECKBOX, label: "Select" });
    sales_sublist.addField({ id: "custpage_tranid", type: serverWidget.FieldType.TEXT, label: "Sales Order Number" }
    sales_sublist.addField({ id: "custpage_entity", type: serverWidget.FieldType.TEXT, label: "Customer Name" });
    sales_sublist.addField({ id: "custpage_trandate", type: serverWidget.FieldType.TEXT, label: "Date" }),
    sales_sublist.addField({ id: "custpage_amount", type: serverWidget.FieldType.TEXT, label: "Amount" });
    sales_sublist.addField({ id: "custpage_status", type: serverWidget.FieldType.TEXT, label: "Status" });

    if(customerName && fromDate && toDate ){



        var salesorderSearchObj = search.create({
        type: "salesorder",
        filters:
            [
```

```
    sales_sublist.addField({ id: "custpage_trandate", type: serverWidget.FieldType.TEXT, label: "Date" }),
    sales_sublist.addField({ id: "custpage_amount", type: serverWidget.FieldType.TEXT, label: "Amount" });
    sales_sublist.addField({ id: "custpage_status", type: serverWidget.FieldType.TEXT, label: "Status" });

    if(customerName && fromDate && toDate ){



        var salesorderSearchObj = search.create({
        type: "salesorder",
        filters:
            [
                ["mainline", "is", true],
                "AND",
                ["entity", "anyof", customerName],
                "AND",
                ["trandate", "within", fromDate, toDate]

            ],

        columns:
            [
                search.createColumn({
                    name: "salesorder",
                    sort: search.Sort.ASC,
                    label: "Sales Order"
                }),
                search.createColumn({ name: "mainline", label: "*" }),
                search.createColumn({ name: "internalid", label: "Internal Id" }),
                search.createColumn({ name: "tranid", label: "Document Number" }),
```

```
                search.createColumn({ name: "mainline", label: "*" }),
                search.createColumn({ name: "internalid", label: "Internal Id" }),
                search.createColumn({ name: "tranid", label: "Document Number" }),
                search.createColumn({ name: "trandate", label: "Date" }),
                search.createColumn({ name: "type", label: "Type" }),
                search.createColumn({ name: "entity", label: "Name" }),
                search.createColumn({ name: "account", label: "Account" }),
                search.createColumn({ name: "amount", label: "Amount" }),
                search.createColumn({ name: "status", label: "Status" }),
                search.createColumn({ name: "currency", label: "Currency" }),

            ]
        });

        // log.debug("salesorderSearchObj result count", searchResultCount);
        var resultSearch = salesorderSearchObj.run().getRange({start:0,end:100});


        resultSearch.forEach(function (result, i) {
            sales_sublist.setSublistValue({
                id: 'custpage_tranid',
                line: i,
                value: result.getValue({ name: 'internalid' })
            });

            sales_sublist.setSublistValue({
                id: 'custpage_trandate',
                line: i,
                value: result.getValue({ name: 'trandate' })
            });
```

```
            sales_sublist.setSublistValue({
                id: 'custpage_trandate',
                line: i,
                value: result.getValue({ name: 'trandate' })
            });

            sales_sublist.setSublistValue({
                id: 'custpage_entity',
                line: i,
                value: result.getText({ name: 'entity' })
            });

            sales_sublist.setSublistValue({
                id: 'custpage_status',
                line: i,
                value: result.getValue({ name: 'status' })
            });

            sales_sublist.setSublistValue({
                id: 'custpage_amount',
                line: i,
                value: result.getValue({ name: 'amount' })
            });

        });
```

```
form.addSubmitButton({
    label: 'Search Sales Order',
});

form.addButton({
id: 'custpage_set_memo_button',
label: 'Set Memo',
functionName: 'setMemo'
});

if(SO_Array){
    log.debug('SO_Array' , SO_Array)
    var scheduledScript = task.create({
         taskType: task.TaskType.SCHEDULED_SCRIPT
     });
      scheduledScript.scriptId = 'customscript815';
      scheduledScript.deploymentId = 'customdeploy2';
      scheduledScript.params = {
            'custscript_so_id_array' : SO_Array
    };
      var scriptTaskId = scheduledScript.submit();
      log.debug("id", scriptTaskId);
}


scriptContext.response.writePage(form);
```

```
}
else if (scriptContext.request.method === 'POST') {

    const delimiter = /\u0001/;
    const customerName = scriptContext.request.parameters.custpage_customer_name;
    const fromDate = scriptContext.request.parameters.custpage_from_date;
    const toDate = scriptContext.request.parameters.custpage_to_date;
    const so_array = scriptContext.request.parameters.custpage_arr;

    scriptContext.response.sendRedirect({
        type: 'SUITELET',
        identifier: 'customscript819',
        id: 'customdeploy1',
        parameters: {
            custpage_customer_name: customerName,
            custpage_from_date: fromDate,
            custpage_to_date: toDate
        }
    });
    if(so_array){
        scriptContext.response.sendRedirect({
        type: 'SUITELET',
        identifier: 'customscript819',
        id: 'customdeploy1',
        parameters: {
            custpage_arr: so_array,

        }
        });
    }
```

2.Client Script

```
/**
 * @NApiVersion 2.x
 * @NScriptType ClientScript
 * @NModuleScope SameAccount
 */
define(['N/currentRecord'],
    /**
     * @param{currentRecord} currentRecord
     */
    function (currentRecord) {

        var so_array = [];


        function pageInit(scriptContext) {


        }

        function fieldChanged(scriptContext) {

            if(scriptContext.fieldId == 'custpage_checkbox'){
                var objRecord = scriptContext.currentRecord

            var line_no = scriptContext.line

            var check_box_select = objRecord.getSublistValue({
                sublistId: 'sales_order_sublist',
                fieldId: 'custpage_checkbox',
                line: line_no
```

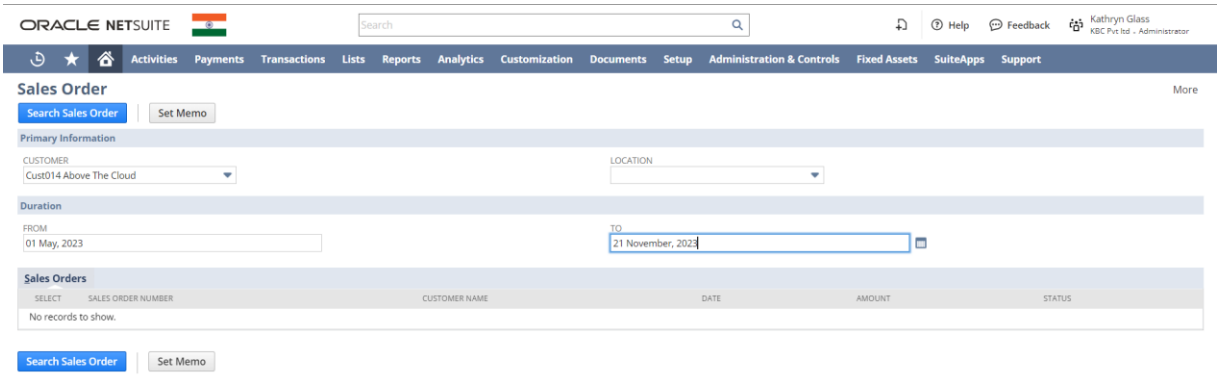
```
                fieldId: 'custpage_checkbox',
                line: line_no
            });

            var sales_order_select = objRecord.getSublistValue({
                sublistId: 'sales_order_sublist',
                fieldId: 'custpage_tranid',
                line: line_no
            });
            so_array.push(sales_order_select)

            }
        }

        function setMemo() {
            var url = 'https://tstdrv2816485.app.netsuite.com/app/site/hosting/scriptlet.nl?script=819&deploy=1&compid=TSTDRV281
            url += '&custpage_arr='+so_array;
            alert('Setting Memo for these sales order :'so_array)
            window.open(url , '_self');

        }

        return {
            pageInit: pageInit,
            fieldChanged: fieldChanged,
            setMemo: setMemo
        };

    });
```

3.Schedule Script

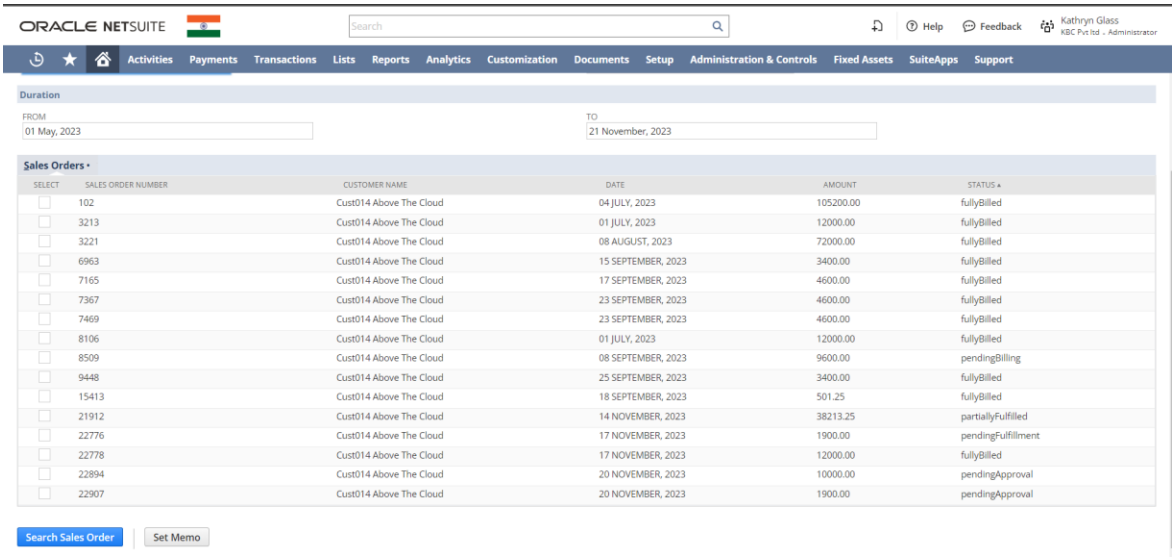
```
/**
* @NApiVersion 2.1
* @NScriptType ScheduledScript
*/
define(["N/log", "N/record", "N/runtime", "N/search"],
    (log, record, runtime, search) => {

        const execute = (scriptContext) => {
            var salesorder_id_array = runtime.getCurrentScript().getParameter({
                name: 'custscript_so_id_array'
            });
            log.debug('Sales Order : ', salesorder_id_array);

            const arr = salesorder_id_array.split(',');

            log.debug('arr : ', arr);

            for (var i = 0; i < arr.length; i++) {
                var a = parseInt(arr[i])

                var salesorder = record.load({
                    type: record.Type.SALES_ORDER,
                    id: parseInt(arr[i]),
                    isDynamic: true,
                })
                salesorder.setValue({
                    fieldId: 'memo',
                    value: 'Update from Schedule Script',
                })
                salesorder.save({
```

```
        log.debug('arr : ', arr);

        for (var i = 0; i < arr.length; i++) {
            var a = parseInt(arr[i])

            var salesorder = record.load({
                type: record.Type.SALES_ORDER,
                id: parseInt(arr[i]),
                isDynamic: true,
            })
            salesorder.setValue({
                fieldId: 'memo',
                value: 'Update from Schedule Script',
            })
            salesorder.save({
                enableSourcing: true,
                ignoreMandatoryFields: true
            });
            log.debug('Memo has been saved ', 'Memo has been saved ' );
        }


    };

    return { execute };
});
```

1.Suitelet form

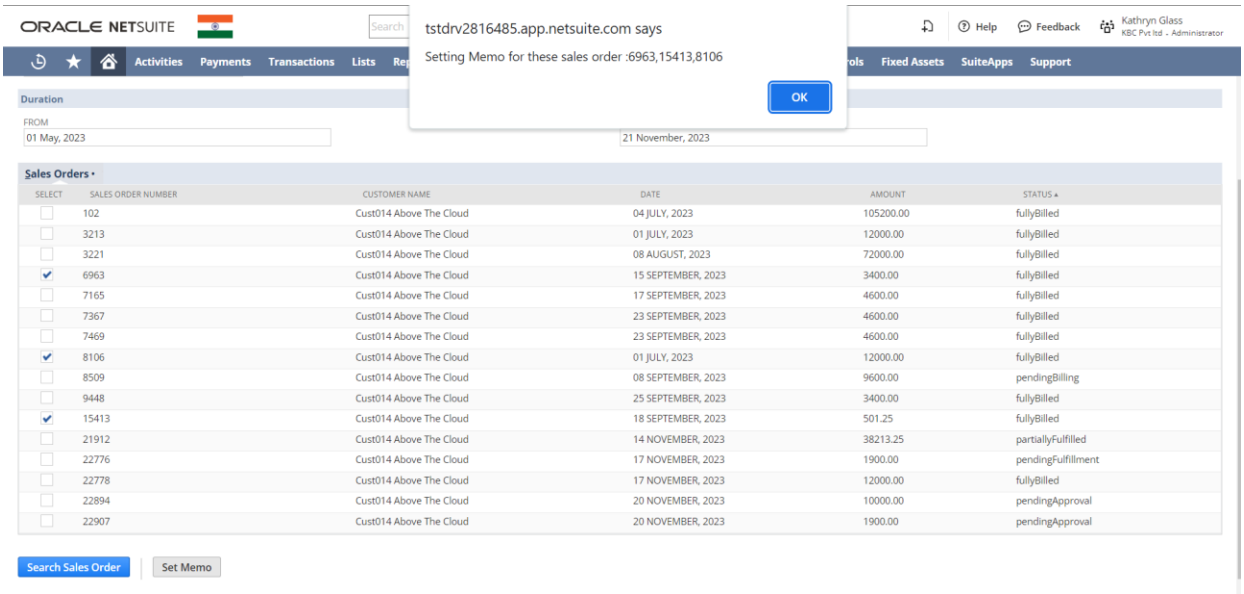


2.Sales order has been search



SELECT	SALES ORDER NUMBER	CUSTOMER NAME	DATE	AMOUNT	STATUS
	102	Cust014 Above The Cloud	04 JULY, 2023	105200.00	fullyBilled
	3213	Cust014 Above The Cloud	01 JULY, 2023	12000.00	fullyBilled
	3221	Cust014 Above The Cloud	08 AUGUST, 2023	72000.00	fullyBilled
	6963	Cust014 Above The Cloud	15 SEPTEMBER, 2023	3400.00	fullyBilled
	7165	Cust014 Above The Cloud	17 SEPTEMBER, 2023	4600.00	fullyBilled
	7367	Cust014 Above The Cloud	23 SEPTEMBER, 2023	4600.00	fullyBilled
	7469	Cust014 Above The Cloud	23 SEPTEMBER, 2023	4600.00	fullyBilled
	8106	Cust014 Above The Cloud	01 JULY, 2023	12000.00	fullyBilled
	8509	Cust014 Above The Cloud	08 SEPTEMBER, 2023	9600.00	pendingBilling
	9448	Cust014 Above The Cloud	25 SEPTEMBER, 2023	3400.00	fullyBilled
	15413	Cust014 Above The Cloud	18 SEPTEMBER, 2023	501.25	fullyBilled
	21912	Cust014 Above The Cloud	14 NOVEMBER, 2023	38213.25	partiallyFulfilled
	22776	Cust014 Above The Cloud	17 NOVEMBER, 2023	1900.00	pendingFulfillment
	22778	Cust014 Above The Cloud	17 NOVEMBER, 2023	12000.00	fullyBilled
	22894	Cust014 Above The Cloud	20 NOVEMBER, 2023	10000.00	pendingApproval
	22907	Cust014 Above The Cloud	20 NOVEMBER, 2023	1900.00	pendingApproval

3.Selected sales order has been saved in array



SELECT	SALES ORDER NUMBER	CUSTOMER NAME	DATE	AMOUNT	STATUS
	102	Cust014 Above The Cloud	04 JULY, 2023	105200.00	fullyBilled
	3213	Cust014 Above The Cloud	01 JULY, 2023	12000.00	fullyBilled
	3221	Cust014 Above The Cloud	08 AUGUST, 2023	72000.00	fullyBilled
✔	6963	Cust014 Above The Cloud	15 SEPTEMBER, 2023	3400.00	fullyBilled
	7165	Cust014 Above The Cloud	17 SEPTEMBER, 2023	4600.00	fullyBilled
	7367	Cust014 Above The Cloud	23 SEPTEMBER, 2023	4600.00	fullyBilled
	7469	Cust014 Above The Cloud	23 SEPTEMBER, 2023	4600.00	fullyBilled
✔	8106	Cust014 Above The Cloud	01 JULY, 2023	12000.00	fullyBilled
	8509	Cust014 Above The Cloud	08 SEPTEMBER, 2023	9600.00	pendingBilling
	9448	Cust014 Above The Cloud	25 SEPTEMBER, 2023	3400.00	fullyBilled
✔	15413	Cust014 Above The Cloud	18 SEPTEMBER, 2023	501.25	fullyBilled
	21912	Cust014 Above The Cloud	14 NOVEMBER, 2023	38213.25	partiallyFulfilled
	22776	Cust014 Above The Cloud	17 NOVEMBER, 2023	1900.00	pendingFulfillment
	22778	Cust014 Above The Cloud	17 NOVEMBER, 2023	12000.00	fullyBilled
	22894	Cust014 Above The Cloud	20 NOVEMBER, 2023	10000.00	pendingApproval
	22907	Cust014 Above The Cloud	20 NOVEMBER, 2023	1900.00	pendingApproval

4.Execution log

Scripts | Parameters | Unhandled Errors | **Execution Log** | Deployments | System Notes

VIEW: Default Script Notes View | TYPE: - All -

Customize View | Remove all | Refresh

#	VIEW	TYPE	TITLE	DATE	TIME	USER	DETAILS	REMOVE
1	View	Debug	Memo has been saved	21 NOVEMBER, 2023	5:37 am	Kathryn Glass	Memo has been saved	Remove
2	View	Debug	Memo has been saved	21 NOVEMBER, 2023	5:37 am	Kathryn Glass	Memo has been saved	Remove
3	View	Debug	Memo has been saved	21 NOVEMBER, 2023	5:36 am	Kathryn Glass	Memo has been saved	Remove
4	View	Debug	arr :	21 NOVEMBER, 2023	5:36 am	Kathryn Glass	["6963","15413","8106"]	Remove
5	View	Debug	Sales Order :	21 NOVEMBER, 2023	5:36 am	Kathryn Glass	6963,15413,8106	Remove

5.Memo has set

Edit | View	6962	15 SEPTEMBER, 2023	Print	SO76	Cust010 Zenith Systems	Billed		US Dollar	$3,400.00	3,400.00
Edit | View	6963	15 SEPTEMBER, 2023	Print	SO77	Cust014 Above The Cloud	Billed	Update from Schedule Script	US Dollar	$3,400.00	3,400.00
Edit | View	6451	14 SEPTEMBER,	Print	SO70	Cust016 Aspen Technologies	Billed		US Dollar	$50,500.00	50,500.00
Edit | View	15410	18 SEPTEMBER, 2023	Print	SO130	Cust015 ABC Co.	Billed		US Dollar	$200.50	200.50
Edit | View	15413	18 SEPTEMBER, 2023	Print	SO131	Cust014 Above The Cloud	Billed	Update from Schedule Script	US Dollar	$501.25	501.25
Edit | View	7165	17 SEPTEMBER,	Print	SO79	Cust014 Above The Cloud	Billed		US Dollar	$4,600.00	4,600.00
Edit | View	3213	01 JULY, 2023	Print	SO18	Cust014 Above The Cloud	Billed		US Dollar	$12,000.00	12,000.00
Edit | View	8106	01 JULY, 2023	Print	SO87	Cust014 Above The Cloud	Billed	Update from Schedule Script	US Dollar	$12,000.00	12,000.00
Edit | View	204	30 JUNE, 2023	Print	SO03	Cust015 ABC Co.	Billed		US Dollar	$86,000.00	86,000.00